Face Recognition Using Eigenfaces: A Linear Algebra Approach

Soheil Hemmat[1] and Hesam Sharifi[1]

[1]Shahed University, Tehran, Iran

Abstract

Face recognition is an important problem in computer vision and pattern recognition. The goal of a face recognition system is to identify or verify individuals from digital images. One of the classical approaches is the Eigenfaces method, based on Principal Component Analysis (PCA). This paper implements the Eigenfaces approach using the Cropped Yale Face Dataset and provides a complete mathematical description, Python implementation details, and experimental evaluation. The system learns from a training set and identifies faces in new test images.

Keywords: Face recognition, Eigenfaces, PCA, dimensionality reduction, pattern recognition.

1. Introduction

Face recognition is an important problem in computer vision and pattern recognition. The goal of a face recognition system is to identify or verify individuals from digital images.

One of the classical approaches for face recognition is the Eigenfaces method [1, 2, 3], which is based on PCA [7]. This method reduces the dimensionality of face images while preserving the most important information required to distinguish between different individuals.

In this project, a face recognition system is implemented using the Eigenfaces approach and the Cropped Yale Face Dataset[1]. The system is capable of learning from a training set of images and identifying faces in new test images.

2. Dataset

We evaluate our method on the Cropped Yale Face Dataset, a widely adopted benchmark for face recognition under varying illumination and expression [6]. The dataset comprises 2,451 grayscale images of 38 subjects, each of size 192×168 pixels, stored in PGM format. Images were captured under controlled laboratory conditions with systematic changes in lighting direction and facial expression, making the dataset particularly suited for assessing the robustness of appearance-based methods such as PCA. Figure 1 shows representative samples.

[1]Dataset available at: https://www.kaggle.com/datasets/jessicali9530/yalefaces

Figure 1: Sample images from the Yale Face Dataset.

2.1. Dataset Organization

Each subject's images are stored in a dedicated subdirectory. We follow the standard evaluation protocol by randomly partitioning the dataset into training (80%) and test (20%) sets, ensuring that all subjects appear in both splits. The directory structure is summarized below:

```
\texttt{CroppedYale/}\\
\texttt{   yaleB01/}\\
\texttt{      yaleB01\_P00A+000E+00.pgm ...}\\
\texttt{   yaleB02/}\\
\texttt{   ...}
```

3. Algorithm

The Eigenfaces method proceeds in several stages, from dataset preprocessing to recognition of new images. The implementation is written in Python, relying on NumPy for linear algebra and eigenvalue

computations, OpenCV for image handling and preprocessing, and Matplotlib for visualization.

3.1. Data Preprocessing

Each subject in the Cropped Yale dataset resides in a dedicated folder. All images are resized to 80×80 pixels, histogram-equalized to mitigate illumination differences, and normalized to the range $[0, 1]$. The code for loading and preprocessing the dataset is shown in Listing 1.

Listing 1: Dataset loading and preprocessing.

```
def load_images(path):
    images = []
    labels = []
    for person in sorted(os.listdir(path)):
        person_path = os.path.join(path, person
            )
        if not os.path.isdir(person_path):
            continue
        for img_name in os.listdir(person_path)
            :
            if not img_name.lower().endswith(".
                pgm"):
                continue
            img_path = os.path.join(person_path
                , img_name)
            img = cv2.imread(img_path, cv2.
                IMREAD_GRAYSCALE)
            if img is None:
                continue
            img = cv2.resize(img, IMG_SIZE)
            img = cv2.equalizeHist(img)
            img = img.astype(np.float32) /
                255.0
            images.append(img.flatten())
            labels.append(person)
    return np.array(images), np.array(labels)
```

After loading, the dataset is randomly split into training (80%) and test (20%) sets, ensuring that all identities appear in both partitions.

3.2. Mean Face and Data Centering

All images are resized to 80×80 pixels, histogram-equalized, and normalized to $[0, 1]$. To apply Principal Component Analysis (PCA), the mean image of the training set is first computed:

$$\bar{x} = \frac{1}{M} \sum_{i=1}^{M} x_i. \tag{1}$$

Each image is then centered:

$$\phi_i = x_i - \bar{x}. \tag{2}$$

The centered images form the matrix:

$$A = [\phi_1, \phi_2, \ldots, \phi_M]. \tag{3}$$

This step removes the global illumination bias and ensures that PCA captures the variations between faces rather than absolute pixel values.

The corresponding implementation is shown in Listing 2.

Listing 2: Computing the mean face and centering the data.

```
mean_face = np.mean(X_train, axis=0)
A_train = X_train - mean_face
A_test = X_test - mean_face
```

3.3. Covariance Matrix, PCA, and Whitening

PCA is used to find the main directions of variation in the face dataset. Let $\mathbf{A}$ be the centered data matrix containing all training images. The covariance matrix of the dataset is defined as

$$\mathbf{C} = \frac{1}{M} \mathbf{A}^\top \mathbf{A}, \tag{4}$$

where M is the number of training images.

Directly computing the eigenvectors of $\mathbf{C}$ is computationally expensive because the dimensionality N of face images is very high. The Eigenfaces method therefore computes the eigenvectors of the smaller $M \times M$ matrix:

$$\mathbf{L} = \mathbf{A}\mathbf{A}^\top. \tag{5}$$

Solving $\mathbf{L}\mathbf{v}_k = \lambda_k \mathbf{v}_k$ yields the eigenvectors $\mathbf{v}_k$ and eigenvalues λ_k. The eigenfaces in the original image space are obtained by back-projection:

$$\mathbf{u}_k = \mathbf{A}^\top \mathbf{v}_k, \tag{6}$$

and each $\mathbf{u}_k$ is normalized to unit length.

To improve discrimination, we apply **whitening** (also known as Mahalanobis whitening or ZCA-like normalization in the reduced space). Let λ_k be the eigenvalue corresponding to the k-th eigenface. A centered image $\boldsymbol{\phi}$ is projected onto the k-th component and scaled by $1/\sqrt{\lambda_k + \epsilon}$:

$$w_k = \frac{\mathbf{u}_k^\top \boldsymbol{\phi}}{\sqrt{\lambda_k + \epsilon}}, \tag{7}$$

where $\epsilon = 10^{-8}$ prevents division by zero. The corresponding Python implementation is shown in Listing 3.

Listing 3: Computing eigenvectors of the reduced matrix, eigenfaces, and whitening.

```
# Reduced covariance matrix and eigen
    decomposition
L = A_train @ A_train.T
eigvals, eigvecs = np.linalg.eigh(L)

# Sort by descending eigenvalue and keep top K
```

```
idx = np.argsort(eigvals)[::-1]
eigvals = eigvals[idx][:K]
eigvecs = eigvecs[:, idx][:, :K]

# Compute eigenfaces and normalize
eigenfaces = A_train.T @ eigvecs
eigenfaces /= np.linalg.norm(eigenfaces, axis
    =0, keepdims=True) + 1e-8

# Whitening: scale projections by inverse sqrt
    of eigenvalues
sqrt_eig = np.sqrt(eigvals + 1e-8)
weights_train = (A_train @ eigenfaces) /
    sqrt_eig
```

3.4. Eigenfaces

After computing the eigenvectors of the matrix L, the actual principal components in the original image space must be obtained. This is done by projecting the eigenvectors $\mathbf{v}_k$ of L back into the high-dimensional image space. The Eigenfaces are computed as

$$\mathbf{u}_k = A^T \mathbf{v}_k, \tag{8}$$

where A is the centered data matrix and $\mathbf{v}_k$ are the eigenvectors of L. Each vector $\mathbf{u}_k$ represents one principal direction of variation in the face dataset.

These vectors are called *Eigenfaces* because, when reshaped into the original image dimensions, they appear as ghost-like face images that capture important facial features such as eyes, nose, and overall lighting patterns. To ensure numerical stability and comparable projections, each Eigenface is normalized to have unit length (see Listing 3).

Each Eigenface captures a dominant pattern of variation in the dataset, and together they form a lower-dimensional subspace known as the *face space*.

The number of retained eigenfaces K is a free parameter. We evaluated several values in the range $[20, 120]$ and observed that $K = 80$ achieved the highest recognition accuracy on a held-out validation subset of the training data. This value was therefore adopted for all subsequent experiments.

3.5. Projection into Face Space

After computing the Eigenfaces, each face image can be represented as a linear combination of these basis vectors. This process is called projection into the *face space*.

Given a centered face image $\boldsymbol{\phi}$, its coordinates in the Eigenface space are computed as

$$w_k = \mathbf{u}_k^T \boldsymbol{\phi}, \tag{9}$$

where $\mathbf{u}_k$ represents the k-th Eigenface. The resulting coefficients w_k are called the *weights* of the face in the Eigenface space.

By stacking these coefficients together, each face image is represented by a weight vector that describes how strongly the face aligns with each Eigenface. This representation significantly reduces the dimensionality of the data while preserving the most important facial variations.

The training images are projected into the face space as shown in Listing 4.

Listing 4: Projecting training images.

```
weights_train = (A_train @ eigenfaces) /
    sqrt_eig
```

3.6. Face Recognition and Threshold Determination

Face recognition is performed by comparing the whitened projection of a test image with those of the training images.

First, the test image is centered by subtracting the mean face and then projected into the whitened eigenface space to obtain its weight vector. The projection incorporates the same scaling factors $\sqrt{\lambda_k + \epsilon}$ used during training:

$$\mathbf{w}_{\text{test}} = \left(\frac{\mathbf{u}_1^\top \boldsymbol{\phi}}{\sqrt{\lambda_1 + \epsilon}}, \frac{\mathbf{u}_2^\top \boldsymbol{\phi}}{\sqrt{\lambda_2 + \epsilon}}, \ldots, \frac{\mathbf{u}_K^\top \boldsymbol{\phi}}{\sqrt{\lambda_K + \epsilon}} \right). \tag{10}$$

The similarity between faces is measured using the Euclidean distance between their weight vectors in this whitened space:

$$d = \|\mathbf{w}_{\text{test}} - \mathbf{w}_{\text{train}}\|_2 . \tag{11}$$

The training image with the smallest distance to the test image is considered the closest match. To handle cases where the test face does not belong to any known individual, a distance threshold is employed. Rather than setting an arbitrary value, the threshold is determined automatically from the training data. For each training image, we compute its distance to the nearest training neighbor (excluding itself). If that neighbor shares the same identity, the distance is recorded as an *intra-class distance*. The threshold is then set to the 95^{th} percentile of all intra-class distances, ensuring that 95% of genuine matches fall below the threshold. Test images whose minimum distance exceeds this threshold are classified as *unknown*.

The implementation of the recognition step is shown in Listing 5.

Listing 5: Face recognition function.

```
def recognize(img_vector):
    phi = img_vector - mean_face
```

```
    w = (phi @ eigenfaces) / sqrt_eig

    distances = np.linalg.norm(weights_train -
        w, axis=1)
    idx = np.argmin(distances)

    if distances[idx] > THRESHOLD:
        return "Unknown", idx, distances[idx]
    return y_train[idx], idx, distances[idx]
```

3.7. Evaluation

The performance of the proposed face recognition system is evaluated using recognition accuracy on the test dataset. After training the Eigenface model, each image in the test set is processed by the recognition function and its predicted label is compared with the corresponding ground truth label.

Recognition accuracy measures the proportion of correctly classified test images relative to the total number of test samples. It is defined as

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Test Images}}. \quad (12)$$

A prediction is considered correct when the predicted identity matches the true identity of the test image. By iterating through all test samples and counting the number of correct recognitions, the overall accuracy of the system can be computed.

The corresponding Python implementation is shown in Listing 6.

Listing 6: Computing recognition accuracy.

```
correct = 0
for i in range(len(X_test)):
    pred, _, _ = recognize(X_test[i])
    if pred == y_test[i]:
        correct += 1

accuracy = correct / len(X_test)
```

4. Experimental Results

4.1. Program Output

The program's console output during execution is reproduced in Listing 7. The system achieved an accuracy of 93.08% on the test set.

Listing 7: Program output.

```
Loading dataset...
Total images: 2451
Train images: 1960
Test images: 491
Computing covariance matrix...
Eigen decomposition...
Computing eigenfaces...

Evaluating on test set...
Test images: 491
Correct: 457
Accuracy: 93.08 %

Testing image: testing.pgm

Prediction: yaleB07
Distance: 0.0725
```

The system compares a given testing image with all training images in the Eigenface space. The closest match is determined based on the Euclidean distance between their projected representations. This process allows the system to identify the most similar face in the dataset.

Figure 2: Recognition result: testing image (left) and closest match from the training set (right).

4.2. Eigenfaces Visualization

The Eigenfaces represent the principal components obtained from PCA. Each Eigenface captures a direction of maximum variance in the dataset and highlights important facial features. Figure 3 shows the first five Eigenfaces computed from the training set. Visualizing these components helps in understanding how the model represents facial structures.

4.3. Recognition Accuracy

The performance of the system was evaluated on the test dataset. Out of 491 test images, the system correctly recognized 457 images, resulting in an overall recognition accuracy of approximately 93.08%. This result indicates that the Eigenfaces approach is able to capture the main variations among facial images and achieve reliable recognition performance on the Yale Face Dataset.

A limited number of misclassifications were observed, which are further analyzed in the following section.

Figure 3: First five Eigenfaces computed from the training set.

5. Comparison with Baseline Methods

We compare the Eigenfaces approach with earlier pixel-based methods.

5.1. Baseline Methods

Prior to subspace techniques such as PCA [1], face recognition typically compared raw pixel vectors via Euclidean distance:

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2. \tag{13}$$

Such methods suffer from high dimensionality, sensitivity to illumination and noise, and a lack of meaningful feature representation [4].

5.2. Quantitative Comparison

Table 1 reports recognition accuracy on the Yale Face Dataset.

5.3. Analysis of Improvement

The Eigenfaces method significantly outperforms pixel-based baselines due to PCA's compact, informative subspace [1, 4]. Our implementation uses $K = 80$ components, whitening, and an automatic rejection threshold.

Key advantages include:

- **Redundancy removal:** PCA discards correlated pixels, retaining dominant variations.
- **Noise reduction:** Low-variance components (noise) are excluded.
- **Dimensionality reduction:** A K-dimensional space improves distance-based classification.
- **Global structure:** Eigenfaces capture holistic facial patterns.
- **Whitening:** Normalizes feature variance, improving generalization.

Despite this gain, PCA maximizes total variance rather than class separability; methods such as Fisherfaces explicitly separate classes and can yield further improvements [2].

6. Analysis

We examine two aspects of the results: sources of misclassification and the influence of the number of eigenfaces K.

6.1. Misclassification Analysis

Despite strong overall accuracy, some test images were misclassified. These errors stem from variations poorly captured by the linear PCA subspace:

- **Illumination:** PCA models total variance rather than identity-specific variance, so extreme lighting shifts distort projections.
- **Pose and expression:** Non-frontal poses and dynamic expressions introduce geometric distortions outside the linear span of frontal training images.
- **Resolution and blur:** Low-quality images lack fine texture, causing identity overlap in feature space.
- **Out-of-subspace faces:** Images far from the PCA subspace yield high reconstruction error and are often misassigned.

These issues reflect PCA's fundamental limitation: it maximizes total variance, not discriminative variance. Thus, while PCA offers effective dimensionality reduction, it remains sensitive to photometric and geometric variations.

Table 1: Comparison of recognition accuracy.

Method	Feature Space	Accuracy
Raw Pixel Matching	Original pixels	50%–60% (approx.)
Nearest Neighbor (Raw)	Original pixels	55%–65% (approx.)
Eigenfaces (PCA)	Reduced ($K = 80$)	93.08%

6.2. Impact of the Number of Eigenfaces (K)

The parameter K controls the retained variance and directly affects accuracy and generalization. Small K retains only coarse features (underfitting), while large K includes noise components (overfitting). A standard criterion is the cumulative variance ratio:

$$R(K) = \frac{\sum_{i=1}^{K} \lambda_i}{\sum_{i=1}^{N} \lambda_i}. \tag{14}$$

Empirically, we found that K preserving 90%–95% of total variance gave the best balance; performance improved up to this range and then plateaued.

7. Conclusion

We have presented a face recognition system based on the Eigenfaces method, implemented in Python using Principal Component Analysis. The system achieves 93.08% recognition accuracy on the Cropped Yale Face Dataset, demonstrating that Eigenfaces effectively capture facial structure when sufficient training data is available. While the method is computationally efficient and conceptually simple, it remains sensitive to variations in illumination and pose. Nonetheless, Eigenfaces provide a foundational baseline for understanding linear subspace techniques and serve as a reference point for more advanced approaches, including deep learning-based recognition systems.

References

[1] M. Turk and A. Pentland, *Eigenfaces for Recognition*, Journal of Cognitive Neuroscience, 3(1), pp. 71–86, 1991. https://doi.org/10.1162/jocn.1991.3.1.71

[2] P. N. Belhumeur, J. P. Hespanha, and D. J. Kriegman, *Eigenfaces vs. Fisherfaces: Recognition using class specific linear projection*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 19(7), pp. 711–720, 1997. https://doi.org/10.1109/34.598228

[3] Z. Liu, P. Luo, X. Wang, and X. Tang, *Deep Learning Face Attributes in the Wild*, Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2015. https://doi.org/10.1109/ICCV.2015.425

[4] I. T. Jolliffe, *Principal Component Analysis*, Springer, 2002.

[5] Y. Taigman et al., *DeepFace: Closing the gap to human-level performance in face verification*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2014.

[6] A. S. Georghiades, P. N. Belhumeur, and D. J. Kriegman, *From Few to Many: Illumination Cone Models for Face Recognition under Variable Lighting and Pose*, IEEE Trans. Pattern Anal. Mach. Intell., 23(6), pp. 643–660, 2001.

[7] L. Sirovich and M. Kirby, *Low-dimensional procedure for the characterization of human faces*, Journal of the Optical Society of America A, 4(3), pp. 519–524, 1987.